

Exercises for Week 9

1. CPU Matrix Multiplication

Before using the GPU, we will warm up with a CPU exercise to get used to Numba and JIT compiled code. In this exercise, we will explore implementations of matrix multiplication, checking the performance of each implementation and how it is related with how the CPU caches. Consider the following Python implementation of matrix multiplication:

```
1 import numpy as np
2
3 def matmul(A, B):
4     C = np.zeros((A.shape[0],B.shape[1]))
5     for i in range(A.shape[0]):
6         for j in range(B.shape[1]):
7             for k in range(A.shape[1]):
8                 C[i,j] += A[i,k] * B[k,j]
9     return C
```

1. Using the `@jit(nopython=True)` decorator, add Numba JIT compilation to the above function.
2. Measure the wall time of the original and JIT compiled version for two 100×100 matrices. How much faster is the JIT compiled version? Remember to run the JIT compiled version once before timing so it can compile.
3. **Autolab** Assuming A, B and C are stored row-wise, the innermost loop (over k) is not accessing B in a cache efficient manner. Make a new version of `matmul` where you re-order the loops such that access is cache efficient.
4. Measure the performance of your optimized version for $N \times N$ matrices, where N is at least 200. What do you observe?
5. (Optional) Measure the performance of your optimized version in MFLOP/s for $N \times N$ matrices where N goes from 20 to 2000. Plot the performance against the size of the matrix in kilobytes. Run the measurement as a batch job where you specify a CPU model so your results are repeatable. Mark the sizes of each cache level on your plot. What do you observe?
Hint: the matrix multiplication uses N^3 FLOPs.
Hint: recall how you measured performance from the exercises in week 3.

2. CUDA Vector Addition

1. **Autolab** Make a CUDA kernel `add_kernel` that takes two vectors x and y as input and returns a new vector a where $a_i = x_i + y_i$, i.e., a is the element-wise sum of x and y .
2. Write a Python program that measures the run time of your kernel with two random vectors. Each input vector should be a NumPy array of length 1,000,000.
Hint: Run the kernel once before timing so it has been JIT compiled.
3. **Autolab** Run your timing program as a batch job so results are repeatable. Use the queue `c02613`.
4. Repeat your time measurements where you ensure the arrays reside in pinned memory before passing them to your CUDA kernel. What do you observe and why?
Hint: To get pinned memory, use `numba.cuda.pinned_array`, [numba.cuda.pinned_array_like](#) or [numba.cuda.pinned](#).
5. Repeat your time measurements where you ensure the arrays reside in GPU memory before passing them to your CUDA kernel. What do you observe and why? Based on your results, how much time is spent on data transfer vs. computation?

3. CUDA Matrix Multiplication

1. Make a CUDA kernel `matmul_kernel(A, B, C)` that multiplies two matrices together. Let A and B be the input matrices and C be the output matrix you write to. You may assume the dimensions of A and B match so they can be multiplied. Structure your kernel such that you use a 2D compute grid where thread (i, j) computes the value of $C[i, j]$.
Hint: use `cuda.grid(2)` to get 2D grid position of the current thread.
2. Measure the run time of your kernel for A and B being 2048×2048 matrices and use 16×16 threads per block. Compare the run time with NumPy (e.g., by timing `A @ B`). What do you observe?
3. Repeat your measurements where the matrices reside pinned and device memory. What fraction of time is spent on memory transfers to and from the GPU with and without pinned memory?
4. Repeat your measurements using 256×1 and 1×256 threads per block. Let the matrices be in GPU memory before passing them to the kernel. What do you observe and why? Run your measurements as a batch job so results are repeatable.